



École Polytechnique Fédérale de Lausanne

Distributed transactions
ensuring deconfliction of long range drone flights

by Mariem Baccari

Master Thesis

Pierluca Borsò-Tan
EPFL Supervisor

Michael Barroco
Industry Supervisor

Decentralized and Distributed Systems Laboratory
School of Computer and Communication Sciences
Orbitalize

February 27, 2026

Acknowledgments

I would like to express my heartfelt gratitude to everyone who supported me throughout my master thesis project.

First, I would like to thank my academic and industry supervisors, Pierluca Borsò-Tan and Michael Barroco, for their invaluable guidance and mentorship during this journey.

I am also deeply grateful to my colleagues at Orbitalize: Max, Mike, Michel and many others whose support and advice created an incredible work environment.

Most importantly, this thesis marks the final chapter of my journey at EPFL which would not have been possible without the unwavering support of those who were here at every milestone. I am especially thankful to Maëlys, Mila, Emma, Nicolas and Kilian for their constant encouragement. Lastly, to my family, thank you for your patience and for being my greatest source of motivation.

Lausanne, February 27, 2026

Mariem Baccari

Abstract

This thesis proposes a Raft-based synchronization protocol for the Discovery and Synchronization Service (DSS), a core component of Unmanned Aircraft Systems (UAS) traffic management. We introduce a multi-Raft based design which partitions the airspace into S2 cells, assigns each region to an independent Raft group and uses a two-phase commit protocol to coordinate cross-shard transactions. We assess the viability of this Raft-based solution by evaluating both a single-Raft and the multi-Raft based prototypes. Our evaluation demonstrates that the multi-Raft approach can scale a single-Raft based solution but comes with challenges related to coordination overhead. Overall, this thesis provides insights into the characterization of consensus-based synchronization protocols for the DSS.

Contents

Acknowledgments	2
Abstract	3
1 Introduction	6
2 Background	8
2.1 Unmanned Aircraft System Traffic Management	8
2.1.1 The Discovery and Synchronization Service	8
2.1.2 The InterUSS Platform	9
2.2 Synchronization in Distributed Systems	9
2.2.1 Consensus Algorithms	9
2.2.2 Distributed Databases	11
2.3 The InterUSS DSS Implementation	13
2.3.1 Architecture	13
2.3.2 S2 Cells	13
2.3.3 CockroachDB Migration and Previous Work	14
3 Problem Statement	16
4 Design	18
4.1 System Model	18
4.2 Geographical Sharding and Replication	19
4.3 Request Routing	20
4.4 Cross-Shard Transaction Coordination	22
4.4.1 Failures and Recovery	23
4.5 Data Model	23
4.6 Limitations	24
5 Implementation	25
5.1 Single Raft Implementation	25
5.1.1 PostgreSQL for Instance-Local Persistence	25
5.1.2 Consensus	25

5.1.3	Networking	26
5.2	Multi-Raft Implementation	27
5.2.1	Shard Definition	27
5.2.2	Transaction Workflow	27
5.2.3	Transaction Coordinator Raft (TC-Raft)	28
5.2.4	Reference Shards and Ownership	28
5.2.5	Implementation Limits	28
6	Evaluation	30
6.1	Experimental Setup	30
6.2	System Throughput and Latencies	31
6.3	Robustness Evaluation	34
6.3.1	Leader crashes	34
6.3.2	Network delays on query latency	35
6.4	Cross-shard transactions	35
6.4.1	Experimental Setup	35
6.4.2	Impact and characterization of cross-shard transactions	36
6.4.3	Breakdown of cross-shard transactions phase latencies	37
7	Related and Future Work	40
7.1	Distributed Databases	40
7.2	Blockchains	40
7.3	Spatial Data storage	41
7.4	Discussion and Requirements	41
8	Conclusion	43
	Bibliography	43
A	Previous Work	47
B	Implementation details	49
B.1	Multi-Raft configuration file	49
B.2	Single-Raft parameters	50
B.3	Proposals	50
B.4	gRPC Consensus Service	51
B.5	gRPC Coordinator Service	51
B.6	gRPC Owners Service	52

Chapter 1

Introduction

The integration of Unmanned Aircraft Systems (UAS) into the airspace is becoming increasingly important. UAS have found applications in various industries such as logistics, agriculture and emergency response as well as for personal use. With this growing demand comes a need for effective traffic management. This is where the Unmanned Aircraft System Traffic Management (UTM) ecosystem comes into play. UTM aims to facilitate the exchange of airspace information between the various actors such as drone operators, service providers and authorities.

The Discovery and Synchronization Service (DSS) is a major component of the UTM ecosystem allowing UAS Service Suppliers (USSs) to discover and synchronize each other's operational data. It functions as a distributed registry of information with an external interface defined by the ASTM F3548-21 standard. However, the internal synchronization mechanism is left unspecified, leaving a gap that needs to be addressed.

The current open-source DSS implementation developed by InterUSS relies on CockroachDB to provide replication, fault tolerance and strong consistency. This approach has greatly simplified development in the past by delegating synchronization to an existing distributed database. However, licensing changes have introduced major operational costs, motivating an investigation for a suitable replacement. Multiple existing papers and systems address various aspects of synchronization in distributed systems but there is an additional need to propose a comprehensive design tailored to the DSS and the nature of the DAR.

This thesis proposes and evaluates a Raft-based synchronization protocol for the DSS. In particular, it presents a multi-Raft design based on the geographical sharding of the airspace to ensure the reliability and efficiency of DSS synchronization. The effectiveness of this approach is assessed against a single-Raft implementation, providing insights into the potential of consensus-based synchronization protocols for the DSS.

Using multi-Raft based synchronization offers the strong consistency guaranteed required by the

standards. Moreover, decentralizing the synchronization process mitigates the risks associated with a single point of failure. Finally, this protocol will set the foundation for a standardizable synchronization mechanism, fostering interoperability among implementations.

The key contributions of this thesis are:

- The development of a multi-Raft synchronization protocol for the DSS.
- A comparative analysis of the proposed multi-Raft implementation against a single-Raft baseline to highlight performance improvements.
- Recommendations for future implementations and / or designs of the DSS.

Our results indicate that the multi-Raft approach improves the single-Raft performance but that this progress can be limited by the coordination overhead compared to a single-Raft implementation. These findings showcase the potential and challenges of consensus-based protocols in providing synchronization for the DSS.

Chapter 2

Background

2.1 Unmanned Aircraft System Traffic Management

Unmanned Aircraft System Traffic Management (UTM)[1] is an ecosystem that enables the integration of Unmanned Aircraft Systems (UAS) into the airspace. The UTM framework allows various actors (drone operators, service providers, and authorities) to exchange real-time airspace and operational information.

This communication is done using a decentralized network where coordination occurs between independent UAS Service Suppliers (USSs). Each USS acts on behalf of its operators to share relevant information with peer USSs. This data exchange is facilitated by the Discovery and Synchronization Service (DSS).

2.1.1 The Discovery and Synchronization Service

The Discovery and Synchronization Service is a federated system that enables USSs to discover relevant peers and synchronize operational data. The DSS itself does not calculate trajectories or store specific data. Rather, it functions as a registry that allows the USSs to exchange data directly between each other.

The properties and technical requirements of the DSS are outlined in the ASTM F3548-21[2] Standard Specification for UAS Traffic Management (UTM) UAS Service Supplier (USS) Interoperability. In this standard, data exchanged between USSs or between USSs and the DSS is referred to as an entity. Operational intents and constraints are two types of entities and USSs can create subscriptions to receive information about relevant entities. Operational intents consist in a 4D Volume (space and time) representation of the intent for a UAS operation while constraints consist

in 4D Volumes that inform about specific geographically and time-limited airspace information.

ASTM F3548-21 defines the interfaces that the DSS exposes to USSs to allow the creation and manipulation of operational intent or constraint entity references, querying geographic areas and time ranges of interest and managing subscriptions.

While the DSS is a decentralized system, the standard does not define a specific synchronization protocol for the DSS but requires that the DSS provides a strongly consistent view of the DAR to the USS clients. To ensure that these requirements are met, the DSS requires a mechanism for maintaining agreement on the state of the DAR, naturally leading to the use of consensus protocols (see section 2.2.1) to provide strong consistency across the system.

2.1.2 The InterUSS Platform

The InterUSS Platform[3] is a Linux Foundation project that provides an open-source implementation of the DSS. The current state of the InterUSS DSS will be discussed in more detail in Section 2.3.

2.2 Synchronization in Distributed Systems

The synchronization issue eventually boils down to systems agreeing on the same view and providing a form of fault tolerance. There are various abstractions that implement this and they are all based on consensus algorithms.

2.2.1 Consensus Algorithms

Consensus algorithms allow a set of distributed processes to agree on a single value or a totally ordered sequence of values. As defined in C. Cachin, R. Guerraoui, and L. Rodrigues' "Introduction to Reliable Distributed Programming" book, a crash fault tolerant consensus protocol satisfies the following properties[4]:

- Termination: Every correct process eventually decides some value.
- Validity: If a process decides v , then v was proposed by some process.
- Integrity: No process decides twice.
- Agreement: No two (correct) processes decide differently.

Fault tolerance

Consensus protocols can be classified depending on the fault model that they tolerate:

- Crash fault tolerance (CFT): Processes may fail by crashing and stopping. CFT consensus ensures correctness as long as a majority of nodes (i.e., $\lceil n/2 \rceil$) are up. Such protocols include Paxos[5] and Raft[6] which are used in numerous production systems. Google's Spanner[7] database uses Paxos groups for its replication layer. More recent distributed databases such as CockroachDB[8] and YugabyteDB[9] use Raft.
- Byzantine fault tolerance (BFT): Processes may behave arbitrarily which includes malicious behaviors. BFT consensus requires a total number of nodes $n \geq 3f + 1$ to tolerate f Byzantine faulty nodes. These algorithms are significant in blockchain deployments[10] due to their adversarial environment and the need for trust without reliance on a third party. Some examples of BFT consensus protocols include PBFT[11] and Hotstuff[12].

Leader-based and leaderless consensus

Leader-based consensus algorithms such as Raft[6] rely on a designated leader to coordinate agreement, whereas leaderless algorithms like Paxos[5] allow nodes to collaboratively reach consensus without centralized control. Leader-based protocols can offer lower latency since only the leader manages coordination, but they can eventually become bottlenecks and require leader elections upon failure. Leaderless approaches avoid single points of failure and distribute load more evenly at the cost of increased message complexity and higher latency.

The Paxos Family

Paxos was introduced by L. Lamport in 1998 in "The Part-Time Parliament" article, which provided the original leaderless Paxos[5] algorithm as a foundation for distributed consensus. This protocol defines a two-phase process involving proposers, acceptors, and learners. In theory, any node could act as a proposer and consensus could be reached without a designated coordinator. In practice, concurrent proposals lead to repeated conflicts and high number of messages overhead. To address this inefficiency, later implementations use a leader.

Over the years, multiple papers have proposed extensions and optimizations to the basic Paxos algorithm. Fast Paxos[13] reduces the number of message delays required to reach consensus by allowing clients to send proposals directly to acceptors, at the cost of increasing the complexity of conflict resolution. Egalitarian Paxos (EPaxos)[14] eliminates the need for a leader by allowing any replica to propose independent commands concurrently. CASPaxos[15] tries to simplify Paxos and Raft by aiming to replicate state instead of log commands.

Raft

Raft[6] was introduced in 2014 by D. Ongaro and J. Ousterhout in the article "In Search of an Understandable Consensus Algorithm" as a more understandable alternative to Paxos. The authors achieve this by using a leader-based structure and by separating leader election and log replication. In Raft, each replica is either a "leader" or a "follower". The leader is elected and sends heartbeats to its followers periodically. It keeps their logs replicated and consistent by deciding on new entries' placement in the log. Raft divides the algorithm in two issues:

- **Leader election:** When followers stop receiving heartbeats, they wait for a randomized election timeout before becoming "candidates" to new leader elections. However, a candidate can't win an election until its log contains all committed entries. This restriction ensures state machine safety: If a server has applied a particular log entry to its state machine, then no other server may apply a different command for the same log.
- **Log replication:** The leader takes client requests in forms of commands to be executed by the replicated state machines. The leader then appends the request to its log as a new entry then forwards it to the followers, retrying until all of the followers have stored it. Once the majority of the followers have confirmed, the leader applies the entry to its local state machine and the request is considered committed. The followers then apply the entry to their own local state machines upon learning about the commit.

QuePaxa

QuePaxa[16] was introduced in 2023 in "Quepaxa: Escaping the tyranny of timeouts in consensus" by P. Tennage et al. as a leaderless consensus algorithm designed to overcome the limitations of traditional leader-based protocols like Raft. In particular, it addresses the issues of reliance on timeouts for liveness under adverse or asynchronous network conditions and the lack of robustness facing network attacks or degradations such as denial of service attacks and routing issues by leveraging randomized, asynchronous consensus. It uses a scheduling mechanism to ensure high availability even under worst-case network conditions without sacrificing efficiency under normal conditions.

2.2.2 Distributed Databases

Distributed databases aim to provide scalability, fault-tolerance and replicated data management while maintaining strong consistency. They typically partition the data across nodes or geographical regions and use consensus protocols such as Raft or Paxos to replicate the data. The main design challenge is ensuring global linearizability of transactions without sacrificing performance.

CockroachDB

CockroachDB is a distributed SQL database built on a transactional, key-value store. It partitions the keyspace into contiguous segments called ranges and each range is independently replicated and managed by its own Raft group consisting of three or more replicas.

CockroachDB introduces non-voting replicas in the algorithm. They do not participate to the votes as requiring all replicas to participate to the quorum increases the write latency. These replicas can serve follower reads which are performed on the nearest replica and are used in multi-region deployments to reduce read latencies.

Each Raft group has a leaseholder replica which is responsible for handling all reads and coordinating writes. The leaseholder is often colocated with the Raft leader. Leaseholders must maintain a liveness heartbeat to keep their lease. If a node becomes unreachable or fails to heartbeat, its lease is transferred to another healthy replica via a new Raft leader election. This mechanism ensures fault-tolerant range access and fast recovery from failures or network partitions.

To optimize performance and locality, CockroachDB periodically evaluates workload distribution and may rebalance leases across replicas, allowing leaseholders to migrate closer to the regions generating the most requests, improving read latency and load distribution.

Spanner

Spanner[7] is a globally distributed relational database developed by Google that provides linearizability and high availability across geographically distant datacenters. Spanner partitions its data into shards which are each managed by a Paxos group supporting long-lived leaders.

Spanner manages to provide linearizability using the TrueTime API, which provides globally synchronized timestamps with bounded uncertainty. Transactions in Spanner obtain commit timestamps that respect causality by waiting out this uncertainty interval. This allows Spanner to assign globally consistent timestamps to transactions without sacrificing performance or safety. Each transaction in Spanner is coordinated by a leader replica in its Paxos group.

Transactions affecting multiple groups in CockroachDB and Spanner will be discussed as related work in Section 7.

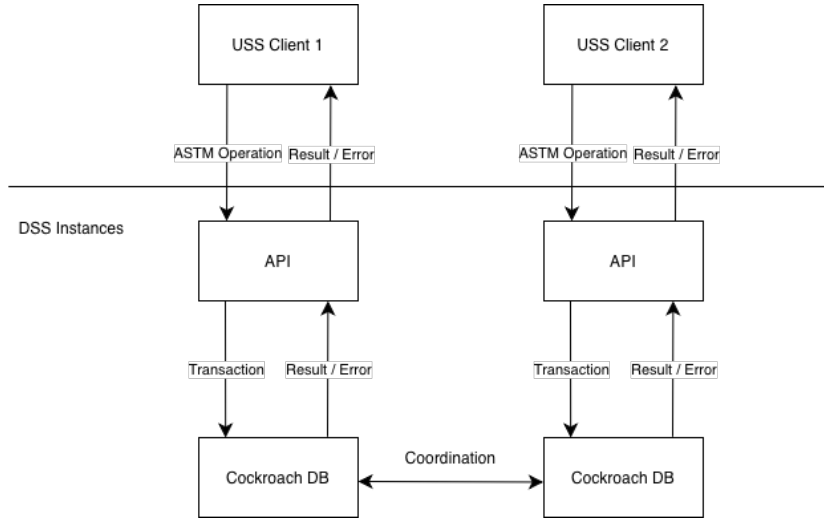


Figure 2.1: Architecture overview of the current InterUSS DSS implementation.

2.3 The InterUSS DSS Implementation

2.3.1 Architecture

Figure 2.1 shows an architecture overview of the current InterUSS DSS implementation. A DSS node exposes a REST API for operations as defined by ASTM F3548-21 that is supported by a CockroachDB instance for the underlying storage and synchronization with other nodes.

2.3.2 S2 Cells

S2 cells [17] are a hierarchical spatial indexing system developed by Google to partition the surface of the Earth into a grid of cells. Each S2 cell is identified by a unique 64-bit known as an `S2CellId` identifier which represents a region on the earth's surface. S2 cells are organized in hierarchical levels, from 0 to 30 with level 30 having the smallest cells and level 0 with the 6 initial cells that then get further subdivided.

In the InterUSS DSS implementation, entities are mapped to S2 cells to manage and query spatial extents. Specifically, the DSS frequently calculates intersections between objects to detect potential conflicts. The S2 cell structure facilitates these tests, ensuring that if two entities intersect, they will share one or more S2 cells [18].

To support this implementation, the Go S2 geometry library [19] provides the necessary representations and methods for converting coordinates to S2 cells and for performing intersection testing.

2.3.3 CockroachDB Migration and Previous Work

In 2024, Cockroach Labs announced a change in licensing terms for new CockroachDB versions causing it to become too expensive to deploy. This goes against InterUSS's goal of promoting easy adoption of UTM services. A substantial investigation [20] has been done by the Technical Steering Committee to find a suitable replacement to CockroachDB which resulted in the short term solution to be using YugabyteDB while an application-level consensus-based solution has been put on hold.

In "CockroachDB mitigation approaches"[21], the TSC identifies the requirements that the DSS implementation should meet. These are detailed in Appendix A. In the same document, the TSC has also identified various high-level alternatives to CockroachDB:

- Although using a different database does not require a large amount of development effort, it maintains the already existing inconveniences of the current implementation such as the blast radius issue with having a single database, having a database that is exposed to the public internet and a lack of clarity in responsibility management as multiple organizations are responsible for a single database.
- A distributed key-value store would eliminate the unnecessary complexities of databases such as CockroachDB and YugabyteDB and conceptually better match the DAR model than a single database. However, this alternative would have the same blast radius, responsibility and exposition issues as a distributed database.
- An application-level consensus solution is the best fit conceptually as it better represents the DSS as a decentralized system instead of a monolithic entity. It also clarifies the boundaries of responsibility and reduces the blast radius. Additionally, a well-defined synchronization protocol can be standardized and improves interoperability and co-operation between different DSS node implementations. The most important issues with this alternative is the large development effort it requires with a high risk of failure and potentially less performant result than an existing solution.

Eventually, migration to YugabyteDB was adopted as a medium-term solution to potentially be replaced with an application-level consensus solution.

Application-level consensus

The TSC identified two candidate algorithms for application-level consensus:

- CASPaxos[15] is a leaderless consensus algorithm aiming to reduce the complexity associated to log replication in other algorithms such as Multi-Paxos and Raft[6] by replicating state. It

was seen as a promising candidate but lacks actively maintained implementations or libraries and implementing it from scratch is not worth the development effort.

- Raft[6] is used by multiple large-scale systems and is implemented in etcd's Raft library in Go which is also used for production systems such as CockroachDB and etcd, making it the most promising and realistic solution for the InterUSS DSS.

Building on the TSC's investigation, we also considered QuePaxa[16] as an alternative for application-level consensus due to its resilience to adverse network conditions. However, QuePaxa introduces some important trade-offs. The protocol is based on asynchronous rounds and hedging strategies and assumes a fixed set of replicas which adds complexity impacting ease of implementation.

On the other hand, this alternative seems to show a tremendous performance improvement compared to Raft under adversarial network conditions affecting the current leader (75k vs 2.5k QPS) and a comparable performance under normal circumstances.

At this stage of the project, we don't consider resistance to adversarial networks as part of the requirements and, therefore, deem Raft an acceptable first solution which, in addition, benefits of a mature and well-documented implementation by etcd [22] that is used in multiple real-world deployments in accordance to our must-have requirement "DSS implementation can be developed and reliably maintained by the available InterUSS contributors."

In December 2024, the TSC published a design document[23] based on Raft for application-level consensus, establishing the overall architecture and milestones for a first phase of the solution consisting in a single Raft group for the entire DSS pool. Considering that such an implementation could cause performance issues due to the leader bottleneck, a second phase including geographical sharding was identified. The phase 2 implementation is expected to support shard coordination to enable horizontal scalability with a Raft group per shard and a "transaction coordination" Raft group which will use two-phase commits for transactions that impact multiple shards.

Chapter 3

Problem Statement

The ASTM F3548-21 standard defines the API for UAS Service Suppliers (USSs) to interact with the Discovery and Synchronization Service (DSS) but does not specify an inter-DSS synchronization protocol. The current InterUSS implementation relies on CockroachDB which provides Raft based replication with sharding and strong consistency as well as multi-region capabilities that automatically handle geo-partitioning. Traditional geo-partitioning aims to keep user data close to the users to improve query latency or to comply with data protection laws by deciding replica placement depending on the origin of the query. This is a generic approach which works regardless of the underlying data but this also means that it does not take into account its nature.

In UTM, the data is inherently geographical: Operational intents, constraints and subscriptions are defined over 4D volumes that the InterUSS DSS implementation maps to S2 cells [18]. This suggests a natural way to partition the data by partitioning the airspace itself and assign responsibility for well-defined geographic regions to independent consensus groups. Such an architecture can offer several advantages over generic geo-partitioning such as reducing cross-partition operations and improving the locality of the queries. Fault isolation can also be improved since failure concerning a specific region would not affect the others. We can also target scaling by assigning more replicas to higher workload regions (Raft groups). On the non technical aspect, we have a better conceptual alignment between the geographical nature of our data and the replication protocol which makes the DSS more intuitive to understand and possibly prepares this architecture to support cross jurisdictional boundaries queries in the context of an international deployment where the link between a jurisdiction and the data it is responsible for is clear. We also eliminate vendor lock-in which kickstarted this investigation and lay the foundation for a standardizable, explicitly defined synchronization protocol for the DSS, fostering interoperability between different implementations.

To address this gap, this thesis proposes a multi-Raft based DSS synchronization protocol in which:

- We divide the airspace into sets of S2 cells to as geographical shards.
- We replicate each shard by an independent Raft group, providing linearizable reads and writes within the shard.
- We handle cross-shard transactions using a two phase commit protocol supported by an independent Transaction Coordination Raft group for transaction state persistence.

Unlike most existing multi-Raft based systems which use geography for replica placement, our design uses multi-Raft in the context of inherently geographical data structures. The goal of this thesis is therefore to design and implement such a system and to evaluate this solution as an optimization to a single-Raft based implementation to characterize the application-level consensus based solution for DSS synchronization.

Chapter 4

Design

This section presents the design of a DSS synchronization protocol based on geographically sharded Raft groups. The design aims to scale throughput by partitioning the data geographically while preserving strong consistency guarantees for all operations, including cross-shard transactions.

4.1 System Model

We consider a system that defines the following components:

- **Discovery and Synchronization Service (DSS):** G independent Raft groups, each responsible for a geographical subset of the airspace and maintaining a strongly consistent replica of the entities related to its assigned region.
- **A Raft group:** $\mathcal{N}_g = \{N_{g,1}, N_{g,2}, \dots\}$ cooperative (DSS) nodes, each storing a replica of their group's area. The nodes may fail by crashing but do not behave maliciously. Upon restart, the nodes can recover from local persistent state. In a Raft group, a maximum of $F_g = N_g/2 - 1$ nodes can be failing at the same time.
- **Transaction Coordination Raft group (TC-Raft group):** A separate Raft group that will persist records of transaction state and decisions to allow transaction completion in the presence of failures.
- **Geographical shard:** A region of the Earth's surface that is defined as a range of S2 cells at a chosen level. Each shard D_g is assigned to a Raft group.
- **UAS Service Suppliers (USSs):** Further referred to as “clients”, they interact with the DSS through the ASTM API. Clients assume strong consistency guarantees and expect identical responses regardless of which DSS node processes the request.

- **ASTM API:** Simple CRUD API, further referred to as "client-server API", defined by ASTM F3548-21 in OpenAPI YAML format. It defines the creation and querying of Operational Intents, Constraints and Subscription objects by the USSs with the DSSs.

We assume that client-server and server-server communications occur over a reliable channel.

4.2 Geographical Sharding and Replication

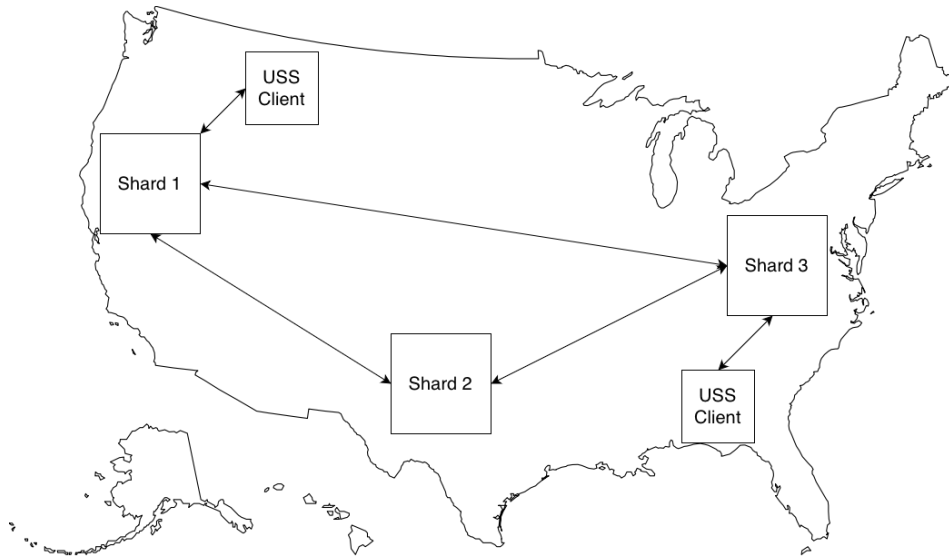


Figure 4.1: Global overview of the system.

We partition the airspace into a fixed set of geographical shards as shown in figure 4.1, defined as sets of S2 cells. Shard definitions and group memberships are static and globally known. This configuration is communicated out-of-band prior to system startup and remains unchanged throughout system operation.

Each geographical shard is replicated by an independent data Raft group as shown in figure 4.2. For cross-shard requests, coordination is required. While we will use a dedicated transaction coordination Raft group, there are other ways to achieve this. The alternatives would be to perform coordination at the server receiving the request or within one of the data Raft groups. However, both approaches complicate failure recovery, as transaction state would not be replicated. Using a Transaction Coordination Raft group (TC-Raft) centralizes coordination logic while simplifying recovery. The primary cost of this design is additional coordination latency and increased load on the TC-Raft group. Each node participates in exactly one data Raft group and the TC-Raft group.

In the context of strategic deconfliction, the DSS manages operational intents, constraints and subscriptions which include geographical extents. These extents are deterministically mapped to S2

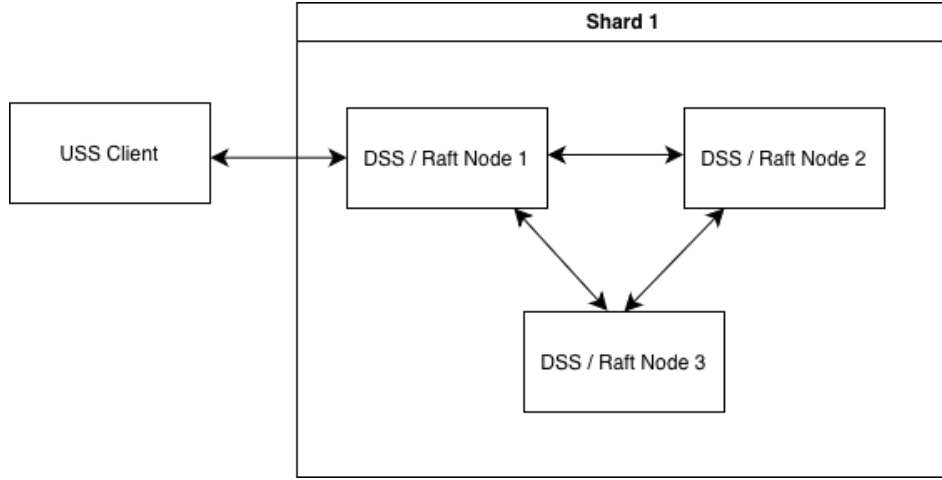


Figure 4.2: Global overview of a shard.

cells, which in turn map to one or more geographical shards.

As a direct cause of using Raft for consensus to order transactions, each data Raft group maintains a strongly consistent replica of all data whose spatial extents intersect with the extents of its shard, providing linearizable reads and writes for all operations in that geographical area.

Each node $N_{g,i}$ uses an instance-local storage system to persist D_g . The storage system should provide durable persistence and atomic operations for the state machine data (operational intents, constraints and subscriptions), high write throughput for serialized writes with efficient snapshotting and restoring.

4.3 Request Routing

Upon receiving a request from a client, a server extracts the geographical extents of the operation and deterministically maps it to a set of S2 cells. Since the shard boundaries are globally known, the server determines the set of shards affected by the request.

If the request affects exactly one shard, the server forwards it to be processed by the appropriate data Raft group. If the request affects multiple shards, it is forwarded to the leader of TC-Raft.

While shard forwarding is derived from geographical extents, some queries from the client-server API (which we consider immutable in the scope of this project) involving, for example, retrieval or deletion do not provide the geographical extents of their objects, only the ID of the object.

Given our current design, this creates a routing ambiguity. Since IDs are randomly generated, they alone cannot allow the server to determine which shard(s) store the object and, therefore, which shard(s) need to be forwarded the query.

Therefore, any correct solution needs to provide a mechanism to resolve object IDs to their owner

shards.

To support these operations, we maintain a mapping from object identifiers to the set of shards that store the object. For each object created, the system deterministically assigns a reference shard using a globally known function: $f : ID \rightarrow ReferenceShardID$. The reference shard is responsible for storing the routing metadata for that object but not the object itself unless it is one of the affected shards. It maintains a replicated key-value state machine mapping object identifiers to shard sets.

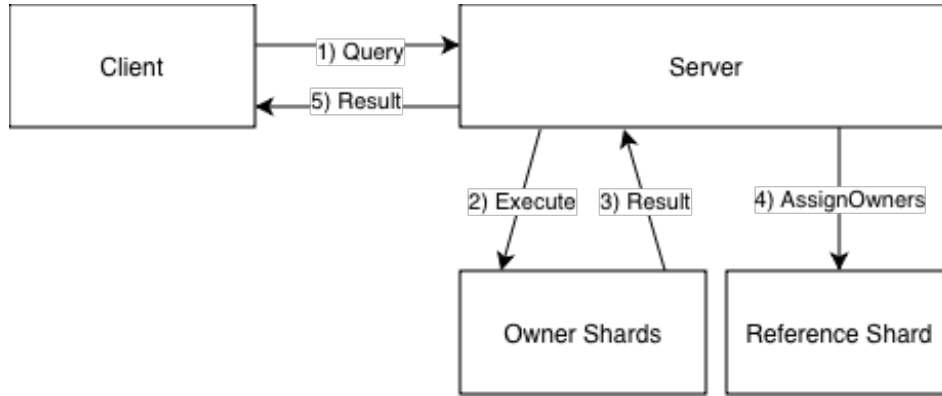


Figure 4.3: Workflow of a creation request.

Figure 4.3 shows the workflow of a creation request. When an object is created, the client provides an object ID and the geographical extents in the request. The server computes the set of affected shards from the extents and the reference shard from the ID. If the reference shard is not one of the affected data shards, routing metadata must be transmitted to it and the processing shard sends a `AssignOwners` to it after the transaction has been committed. The reference shard leader then adds an `AssignOwnerShards` entry to its own Raft log to persist the mapping.

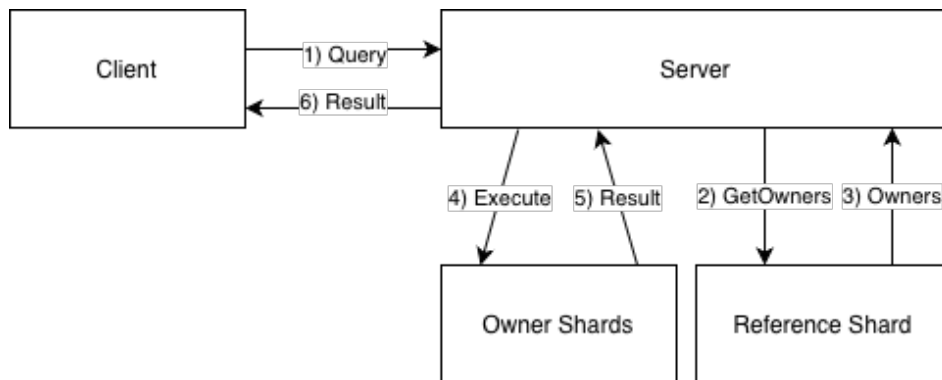


Figure 4.4: Workflow of a query / deletion request.

Figure 4.4 shows the workflow of a query / deletion request. To execute those operations, the receiving server computes the reference shard from the identifier and sends a `GetOwners` request

to the reference shard which performs a linearizable read and returns a set of shards that own the object. The operation can then be routed and executed properly by the server.

The alternatives to this reference shard based design involve centralizing the mapping to the transaction coordination Raft group or broadcasting a request to all shards to gather information about the owners of the objects. However, both introduce a major bottleneck and undermine the scalability efforts that sharding was introduced for.

However, the proposed mechanism still adds a considerable overhead as we now need an additional consensus round for all queries: One for the lookup / reference creation and one for the actual query. Some optimizations such as caching can be implemented to reduce this overhead.

4.4 Cross-Shard Transaction Coordination

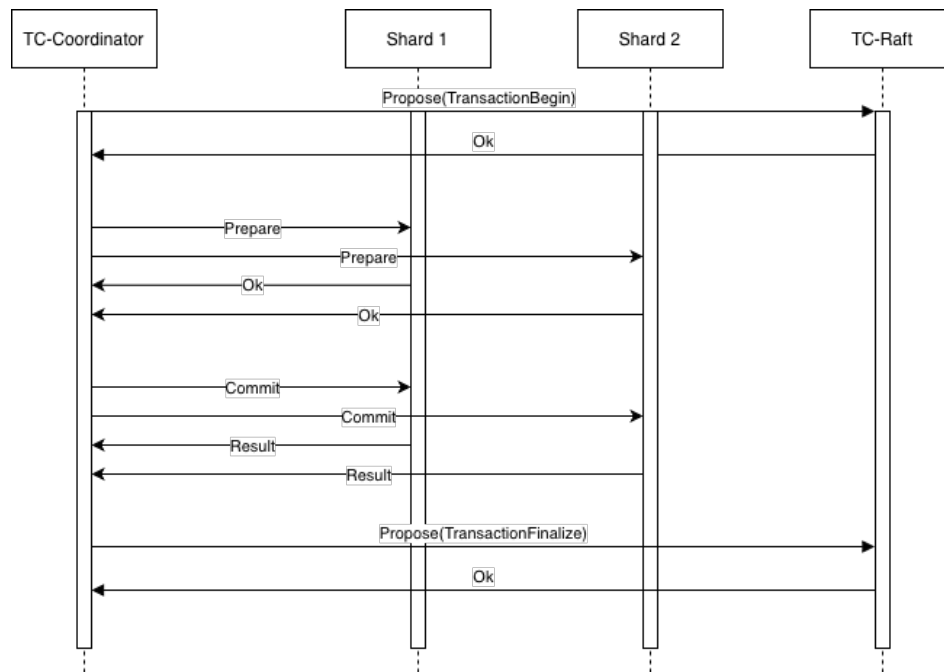


Figure 4.5: Cross-shard transaction workflow.

Cross-shard transactions must be applied atomically across all the affected shards to ensure that all shards commit or abort. Our system uses a two-phase commit protocol coordinated by TC-Raft's leader which allows for durable records of transaction state and decisions, allowing the transactions to complete even in the presence of coordinator failures.

Figure 4.5 shows the transaction workflow of a cross-shard query. When it receives a cross-shard request, the TC-Raft leader (coordinator) initiates the transaction with a `TransactionBegin` proposal to TC-Raft which locks the corresponding shards such that a shard only has one cross-shard

transaction at a given time.

The coordinator then proceeds as follows:

It sends a `Prepare` message to all affected shards containing the proposal. When a shard g receives a `Prepare` message, it validates the request against its local state and initiates a `TransactionPrepare` proposal to $\mathcal{N}_g$. This log entry durably records the pending transaction and causes the shard to prepare the transaction, including acquiring the necessary locks. Once the `TransactionPrepare` entry is committed, the shard responds with any relevant intermediate results, including potential errors.

If all participating shards successfully prepare, the coordinator sends a `Commit` message to all the affected shards which commit the prepared transaction and discards the pending transaction record. If at least one shard fails to prepare, it sends a `Abort` decision instead and shards rollback the transaction.

Finally, the coordinator appends a `TransactionCommit` or `TransactionAbort` entry to its Raft log which removes the shard locks then aggregates the responses received from the shards before returning to the client.

4.4.1 Failures and Recovery

The use of TC-Raft ensures that all critical state transitions are durably replicated and recoverable. If the TC-Raft leader crashes, a new leader is elected and recovers the set of in-progress transactions by replaying the Raft log. The new leader can resume coordination by resending prepare or decision messages. If a shard leader crashes, Raft guarantees that any prepared or committed transactions are preserved and recovered by the new leader. This avoids indefinite blocking and eliminates the dependence on the availability of a specific coordinator node.

4.5 Data Model

This subsection summarizes the contents of the information transmitted described above. Our design distinguishes messages and proposals. A proposal is proposed to a Raft group (data or TC) to modify replicated state (Operation, transaction coordination, ownership assignments) while a message is sent from one server to another to coordinate progress between shards. Messages are not replicated and do not directly modify persistent state.

Before being submitted to Raft, all operations are wrapped into proposals. Each proposal or message includes a globally unique identifier and some data.

A proposal contains:

- An identifier
- A timestamp, determined by the node which first receives the client request
- The proposal type
- The payload
- The set of affected shards

The different proposal types are the following:

Proposal Type	Payload
Data Proposal	An API Operation
AssignOwners	Object ID and list of owner shards
GetOwners	Object ID
TransactionBegin	A data proposal and a list of affected shards
TransactionPrepare	A data proposal
TransactionCommit	ID of the TX proposal
TransactionAbort	ID of the TX proposal

The different types of messages are the following:

Message Type	Payload
Prepare	An API Operation
PrepareResponse	Prepare result
MsgCommit	ID of the TX proposal
MsgAbort	ID of the TX proposal
MsgAck	ID of the TX proposal
MsgAssignOwners	Object ID and list of owner shards
MsgGetOwners	Object ID
MsgOwners	List of owner shards

4.6 Limitations

The reference shard mechanism introduces an additional dependency for ID-based operations and requires an additional consensus round to persist the routing metadata. If a reference shard goes down, operations that depend on it become unavailable preferring correctness over availability.

Chapter 5

Implementation

This section describes the implementation of our DSS prototypes. We first present a single-Raft implementation, which serves both as a performance baseline and as the foundation for key technical decisions regarding storage, networking, and data encoding. We then extend this design to a multi-Raft architecture supporting geographical sharding and cross-shard transactions. Everything is implemented in Go.

5.1 Single Raft Implementation

5.1.1 PostgreSQL for Instance-Local Persistence

We initially considered using SQLite for locally persisting the DAR data due to its embedded and lightweight nature which eliminates deployment overhead. However, SQLite supports a limited set of primitive datatypes which makes the support of UTM needed types difficult and would require additional encoding and decoding logic leading to increased complexity. To avoid these issues, we used PostgreSQL which provides native support for all needed datatypes.

5.1.2 Consensus

For consensus, we use the Raft implementation provided by the etcd project with the parameters in the appendix B.2, including its write-ahead log (WAL) and snapshotting implementations for log persistence. Figure 5.1 explains the end-to-end workflow of a client request.

The consensus package exposes a `Consensus` object with a `ProposeValue()` method, which is invoked by the HTTP API upon receiving a USS client request. Internally, the `Consensus` object embeds an `raft.Node` instance and forwards proposals via the library's `Propose()` method. The

caller blocks until the proposal has been committed and applied.

Committed entries are processed asynchronously by a dedicated goroutine, `handleReady()`, which consumes updates emitted by the Raft node. These updates include committed log entries, configuration changes, and leadership transitions. Once an entry is committed, it is deterministically applied to the state machine.

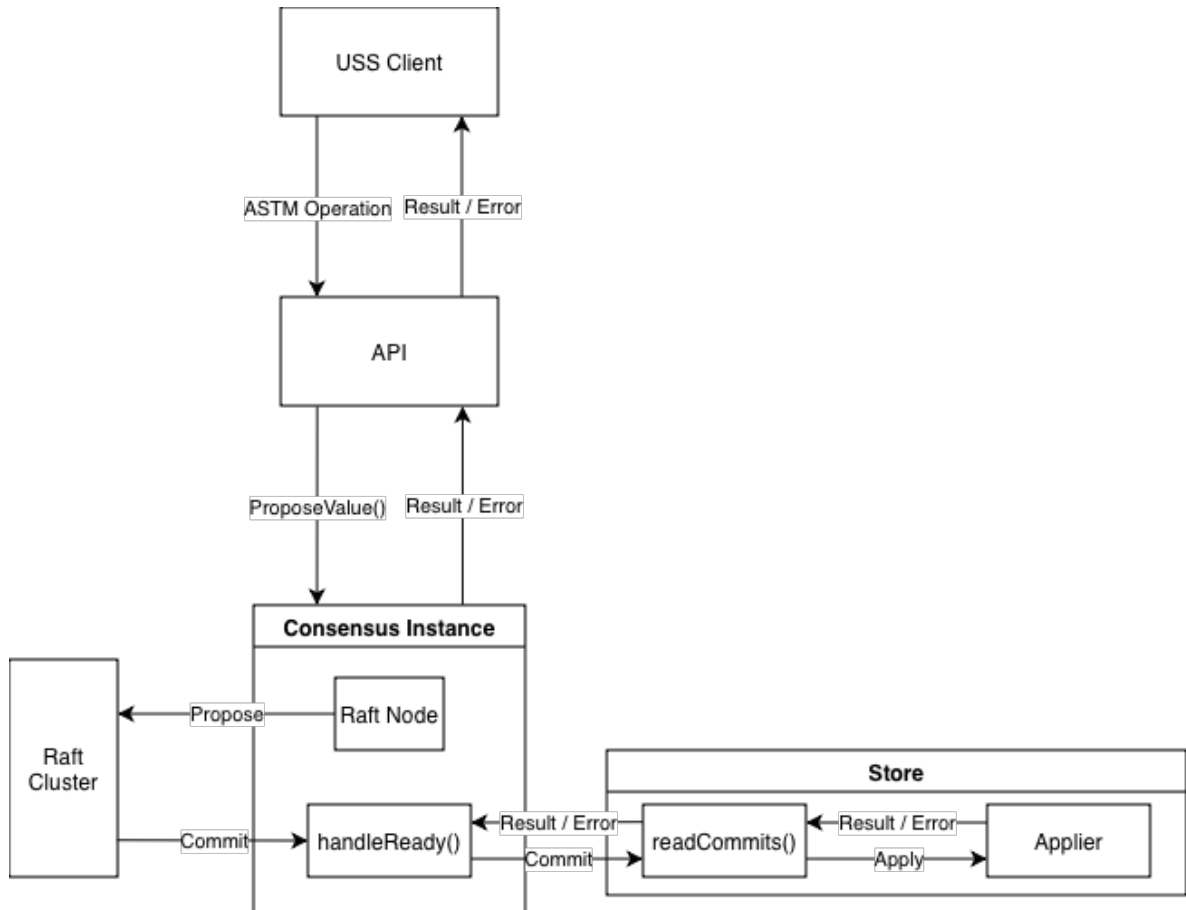


Figure 5.1: Architecture overview of the single-Raft implementation.

Proposals are represented using the structure presented in the appendix B.3: Each proposal is assigned a randomly generated UUID. The timestamp is assigned by the receiving server and is included to ensure deterministic application of time-dependent operations. Proposals are serialized using JSON before being transmitted to other Raft peers.

5.1.3 Networking

Raft requires reliable, ordered message delivery between peers. To satisfy these requirements while minimizing overhead, we implemented a custom transport layer based on TCP and gRPC

streaming as defined in the appendix B.4. Each DSS node exposes a gRPC server implementing the `Consensus.SyncRaft` streaming RPC. For each peer, a single persistent bidirectional stream is established and reused for all Raft messages, avoiding per-message connection setup costs.

During startup, a node initializes its transport, starts the gRPC server, and blocks until it successfully establishes connections to all peers. Connection attempts use an exponential backoff strategy. If a peer cannot be reached after a bounded number of retries, it is marked as inactive. During normal operation, failures due to broken streams cause the peer to be temporarily removed, after which reconnection is attempted.

5.2 Multi-Raft Implementation

The multi-Raft implementation extends the single-Raft design to support geographical sharding and cross-shard transactions coordinated via two-phase commit (2PC). Each DSS node now runs two distinct consensus instances: one for shard-local data replication and one for transaction coordination (TC-Raft).

5.2.1 Shard Definition

At startup, all nodes are provided with the same YAML configuration file defined in the appendix B.1 containing shard IDs, geographical boundaries and shard participants (IDs and addresses). The DSS nodes then use a deterministic `AreaToCellIDs(area string) (s2.CellUnion, error)` function to map the area to S2 cells. For each shard, a node selects one participant as its correspondent. Contacting that node is equivalent to contacting the shard. Further improvements can be made in this area to detect node failures but are left for future work. All inter-shard communication is performed via the gRPC services defined in appendices B.5 for transaction coordination and B.6 for ownership communication.

5.2.2 Transaction Workflow

Figure 5.2 shows the workflow of a transaction. The USS Client interacts with the HTTP API which calls the `Dispatcher`. The `Dispatcher` is responsible for determining the owner shards either by examining the query extents or by contacting reference shards. Then, it either forwards the query to the appropriate shard or to the coordinator in case of a cross-shard transaction which will coordinate the transaction between the determined shards. `Dispatch()` returns a slice of `ShardResult` objects which are then aggregated by the API.

5.2.3 Transaction Coordinator Raft (TC-Raft)

The TC-Raft state machine is stored in memory as a map from a transaction ID to a transaction status and list of affected shards. The possible transitions are `ProposalTypeBegin`, `ProposalTypeCommit` and `ProposalTypeAbort` which are proposed by the coordinator at the beginning and end of each cross-shard transaction.

In principle, persisting transaction state in TC-Raft enables recovery from coordinator failures and leadership changes during transaction execution. However, the current implementation does not fully support these scenarios. Nonetheless, TC-Raft allows us to measure and analyze the latency overhead introduced by cross-shard coordination.

5.2.4 Reference Shards and Ownership

Each object is deterministically assigned a reference shard by computing the sha256 hash of its object ID and applying a modulo operation on the first eight bytes using the total number of shards. This approach ensures a uniform distribution of references across shards.

Reference shards persist ownership metadata in their own data Raft group, mapping object IDs to arrays of owning shard identifiers.

5.2.5 Implementation Limits

The current prototype does not handle coordinator crashes or leadership changes during cross-shard transactions. Coordination responsibility is fixed to the server receiving the initial request, and no explicit shard-level locking is performed, requiring cross-shard transactions over the same shards to be executed sequentially by the clients.

Additionally, failures to persist decisions to TC-Raft or to create object references do not trigger transaction rollback or retries, potentially leading to undefined states in cases where an object is persisted in the Raft group but not in the reference shard map.

Finally, the system does not currently retry dropped proposals which need to be retried by the client. Addressing these limitations is left for future work and they are accounted for in the evaluation which does not trigger such undefined states.

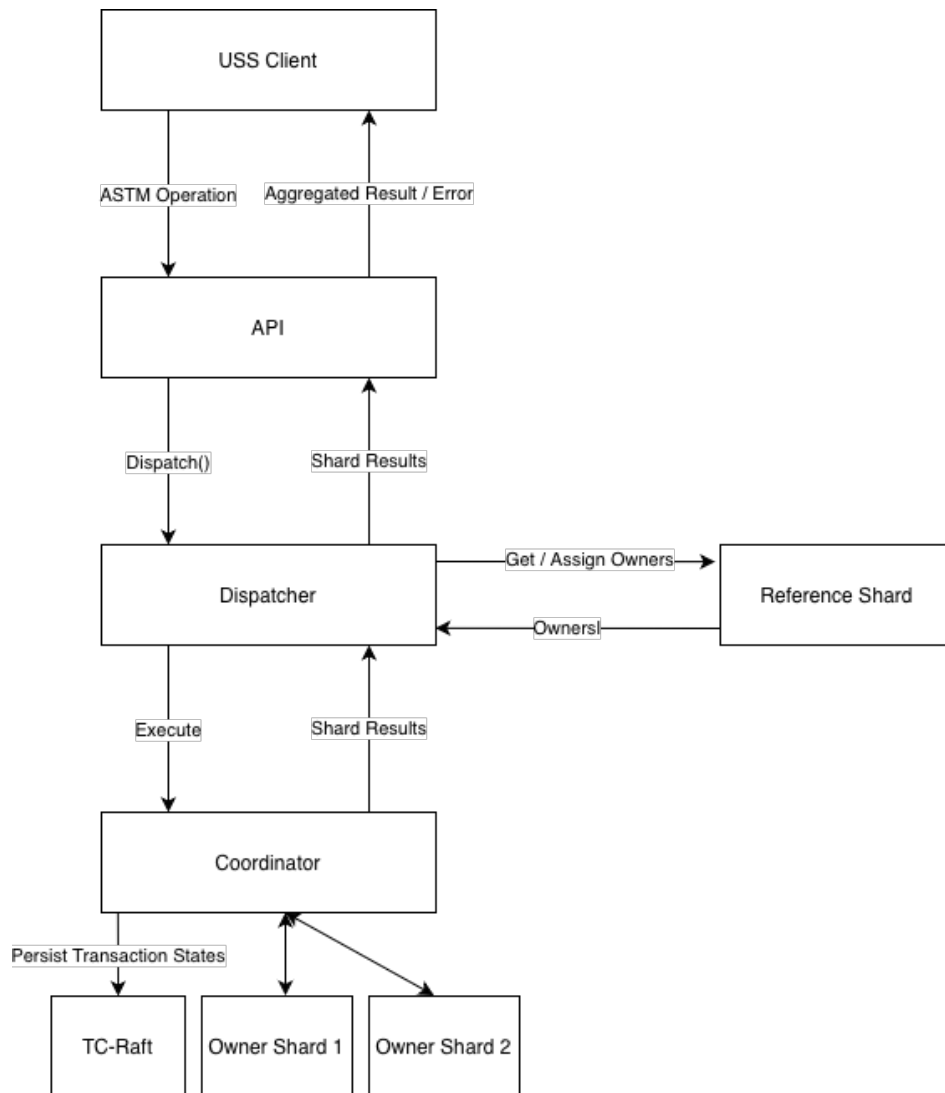


Figure 5.2: Architecture overview of the multi-Raft implementation.

Chapter 6

Evaluation

In this section, we discuss the performance evaluation of the proposed design. We aim to compare the performance of the single-Raft and multi-Raft implementation under different conditions and assess the resource utilization of the system and how the creation of cross-shard transactions impacts the system performance. We also aim to breakdown the transaction phase latencies in order to identify the major bottlenecks.

6.1 Experimental Setup

We deployed the servers on three machines interconnected via a shared local-area network (LAN). All machines ran Ubuntu 24.04.3 LTS (x86_64) with Linux kernel version 6.8.0-90-generic and were each equipped with 2 AMD EPYC-Milan @ 2.399GHz CPUs and 8 GB of RAM.

The servers communicated exclusively over the shared LAN. Round-trip times (RTT) between machines were measured prior to experimentation to characterize network latency. The observed latencies were as follows:

- Server 1 ↔ Server 2: 3 ms
- Server 2 ↔ Server 3: 25.5 ms
- Server 1 ↔ Server 3: 24 ms

Communication delays between the servers are artificially injected in specific experiments.

The clients were executed from a separate machine on the same LAN and with the same configuration. The measured RTTs between the client machine and the servers were the following:

- Client ↔ Server 1: 23.8 ms
- Client ↔ Server 2: 25.4 ms
- Client ↔ Server 3: 0.6 ms

The workload was distributed evenly across the servers. Clients issue "CreateSubscription" HTTP requests with a five seconds timeout (or 15 minutes for cross-shard transactions) to interact with the system as dictated by the DSS performance requirements. To run the clients, we use a Go script which runs a `Generator` per Raft node (3 for single-Raft and 9 for multi-Raft). Each `Generator` spawns goroutines with an `http.Client` each that send a query, waits for the response then either sleeps or sends the next query while aiming to maintain a rythm of roughly one query per second.

For the single-Raft experiments, we deployed a node on each machine. For the multi-Raft experiments, we deploy 3 nodes on each machine such that no Raft group has 2 nodes on the same machine.

6.2 System Throughput and Latencies

In this section, we compare the Single-Raft baseline against the Multi-Raft implementation under single-shard transaction based workload in terms of global throughput and end-to-end latencies to quantify the impact of geo-sharding the system while increasing the coordination costs on overall system performance and to assess how network delays influence the query latency. We also decided to disable reference creation on some of the experiments to assess how the system would behave if we could circumvent this additional mechanism.

We ran these experiments three times each and averaged out the results to minimize the single-experiment variability effect.

Figure 6.4 shows the global system throughput and timed out requests under an increasing number of concurrent clients. The single-Raft deployment contains three nodes while the multi-Raft ones have 3 clusters with 3 nodes each giving us a total of 9 nodes. The single-Raft throughput increases up to 300/400 QPS with a near zero number of failures after which the performance degrades and the system struggles to complete as many requests within the five-second timeout, reaching a ceiling at 500 QPS for 2000 concurrent clients.

The multi-Raft implementation with reference creation reaches a peak of 600 QPS with a near zero number of failures. Considering the fact that this deployment has 3 clusters, we notice the performance deterioration compared to single-Raft. More specifically, each cluster would reach an average maximum of 200 QPS. This is in agreement with our theoretical predictions as we have two consensus rounds per request for this deployment.

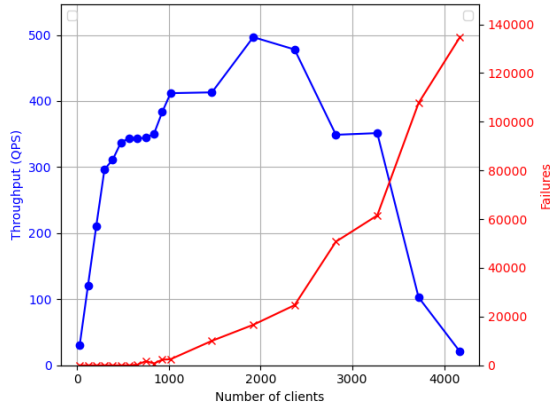


Figure 6.1: Single-Raft

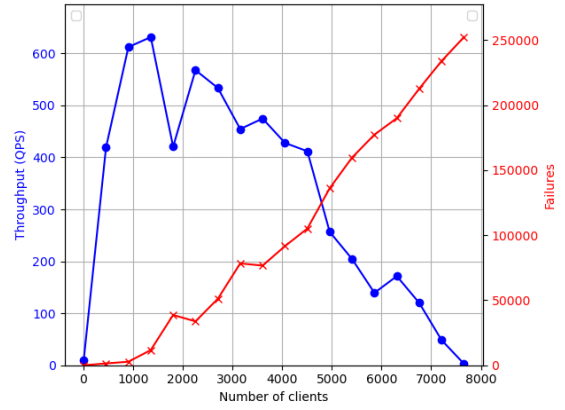


Figure 6.2: Multi-Raft with Reference Creation

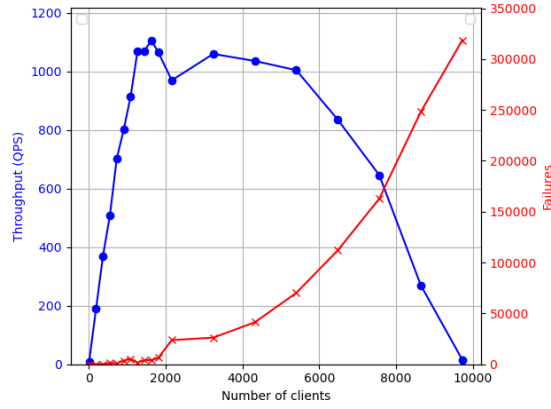


Figure 6.3: Multi-Raft without Reference Creation

Figure 6.4: Global system throughput in queries per second (QPS) under increasing workload for the Single and Multi-Raft implementations.

On the other hand, the multi-Raft deployment without reference creation can reach a peak slightly above 1000 QPS without failures, tripling the single-Raft deployment's throughput as expected.

Figure 6.8 shows the 50th, 90th and 99th latency percentiles during the three experiments. In the single-Raft deployment, latency increases with the number of concurrent clients. As the load increases, the gap between the percentiles widens. This suggests that queuing delays at the leader become more important, amplifying tail latencies. As we reach 2000 concurrent clients, the 90th and 99th percentiles reach the 5 seconds timeout, followed by the 50th percentile. This is coherent with the throughput collapse that we previously noted.

On the multi-raft deployments, we note that, under a manageable workload (under 2000 clients) the average latency for multi-Raft with reference creation queries is double that of the deployment

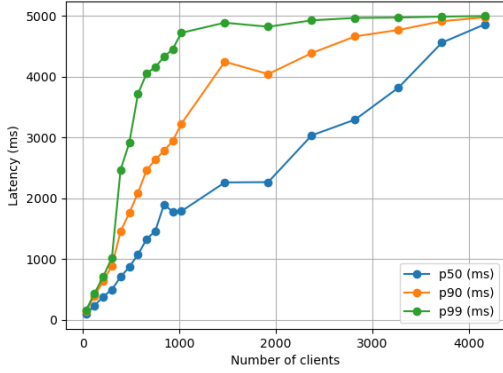


Figure 6.5: Single-Raft

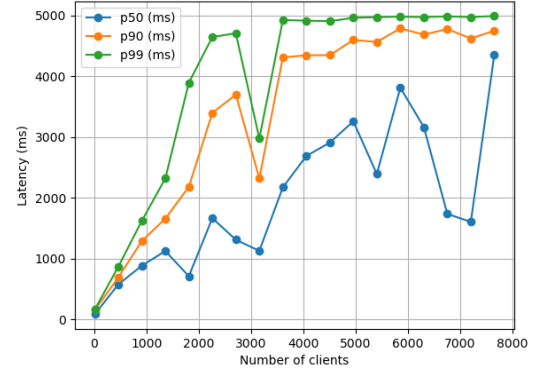


Figure 6.6: Multi-Raft with Reference Creation

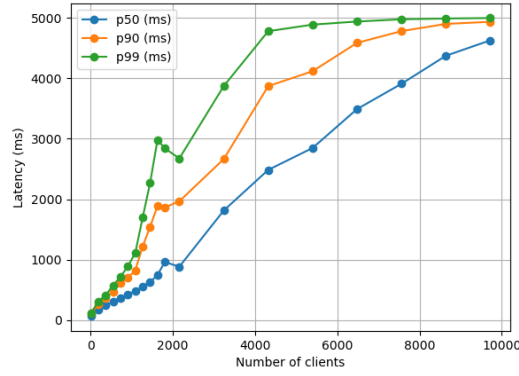


Figure 6.7: Multi-Raft without Reference Creation

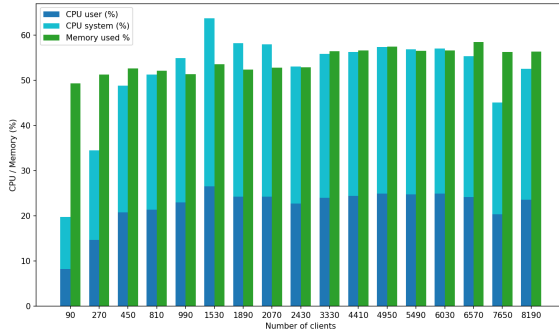
Figure 6.8: Latencies in milliseconds (ms) under increasing workload for the Single and Multi-Raft implementations.

with this feature disabled. This is expected given the doubling of consensus rounds between these two implementations.

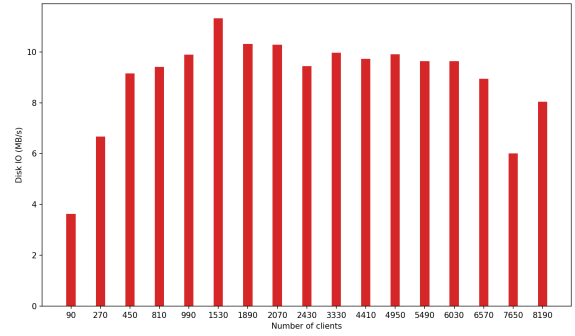
However, the the 90th and 99th percentile curves starting from 4000 clients are quite similar between the two deployments indicating that the cost of a second consensus round is marginal for the slowest queries where queuing acts as the bottleneck.

We notice a sudden drop in average latency at around 7000 concurrent clients in figure 6.6 which could be explained by the small number of experiments that we ran and the variability which comes with that.

Figure 6.9 shows resource utilization metrics under an increasing workload of single-shard transactions on the Multi-Raft (with Reference creation) deployment.



(a) CPU and memory utilization



(b) Disk IO

Figure 6.9: Resource utilization under increasing workload of single-shard transactions on the Multi-Raft deployment.

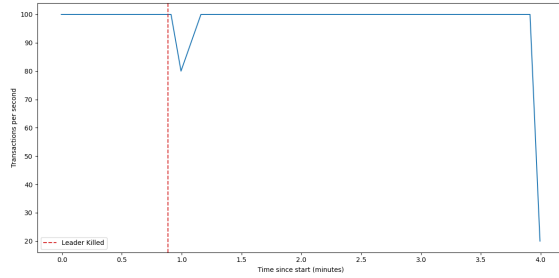


Figure 6.10: Throughput over time with leader crash.

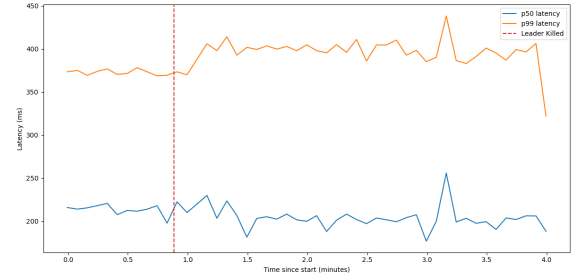


Figure 6.11: p50 and p99 latencies over time with leader crash.

Figure 6.12: Impact of leader crash on performance.

Figure 6.9a shows the CPU and memory utilization while figure 6.9b shows disk IOs. We see an initial increase in CPU and disk IOs reflecting the workload increase. However, the utilization quickly stabilizes and remains the same at higher workloads, suggesting that the bottleneck is not hardware related but rather from application-logic.

6.3 Robustness Evaluation

6.3.1 Leader crashes

Figure 6.12 shows the impact on throughput and latencies of a leader crash when the system is under a usually manageable workload. Figure 6.10 shows that throughput has a 20% decrease and recovers in around 25 seconds. Figure 6.11 shows a noticeable increase in the 99th percentile latency, indicating an increase in queuing time. The fast recovery is essential to meeting the 5th must-have

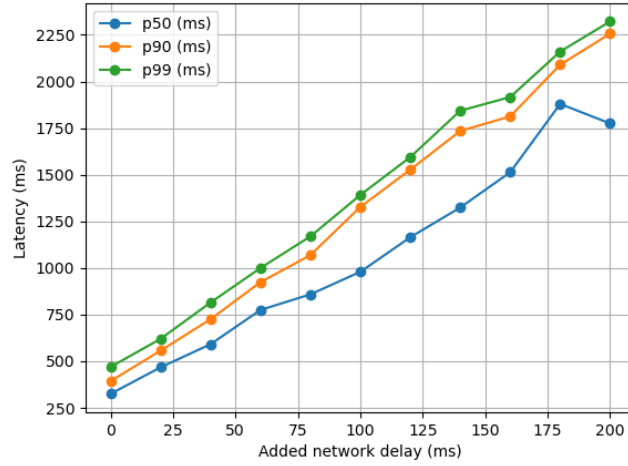


Figure 6.13: Impact of network delays on latency.

requirement in A.

6.3.2 Network delays on query latency

Figure 6.13 illustrates the impact of network delays on latency across percentiles under a manageable workload. In this experiment, we artificially injected a linearly increasing network delay on each of the server’s LAN network interface. We notice a linear increase in latency as network delay increases, showing proportional and stable impact.

6.4 Cross-shard transactions

In this section, we aim to analyze the performance and impact of cross-shard transactions on the system.

6.4.1 Experimental Setup

We maintain the setup previously described and add a new client goroutine responsible for sending a cross-shard transaction with a timeout of 15 minutes every 5 minutes. The client waits for the current query to finish or timeout, sleeps for 5 minutes then fires the next one. The implementation internally imposes a minute long timeout on some operations such as the prepare and the committing.

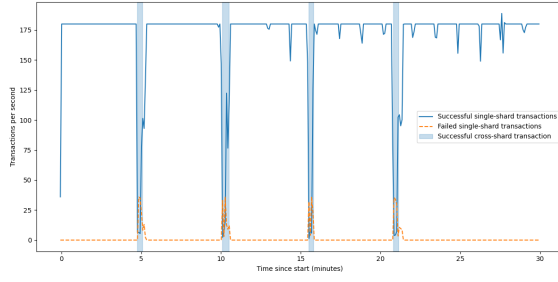


Figure 6.14: Throughput and failures over time with a cross-shard transaction every 5 minutes.

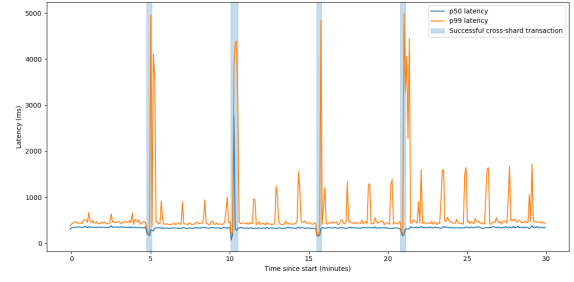


Figure 6.15: p50 and p99 latencies over time with a cross-shard transaction every 5 minutes.

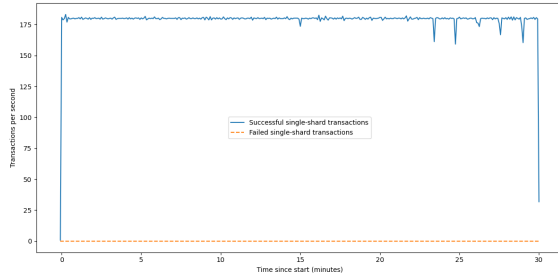


Figure 6.16: Throughput and failures over time without cross-shard transactions.

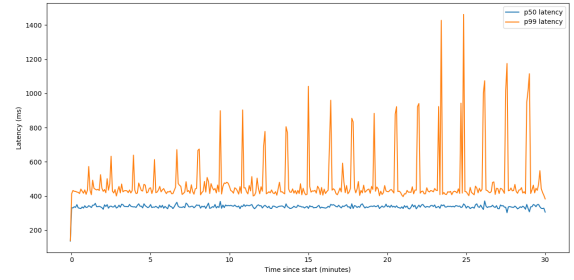


Figure 6.17: p50 and p99 latencies over time without cross-shard transactions.

Figure 6.18: Performance over time with a constant load of 270 single-shard concurrent clients.

6.4.2 Impact and characterization of cross-shard transactions

In order to measure the impact of cross-shard transactions on system performance, the following experiment puts a constant, usually fully manageable, load of 180 concurrent clients sending single-shard transactions with the cross-shard transactions client goroutine described above. Figure 6.18 shows the throughput, failures and latencies over time of single-shard transactions with and without regular cross-shard transactions. Figure 6.14 shows that cross-shard transactions sharply decrease the global system throughput and increase the tail latency during their lifetime. After that, both the throughput and tail latency promptly recover to the baseline numbers. This is coherent with theoretical predictions as the shards are all blocked on processing the given cross-shard transaction, increasing the waiting time for the slowest queries and dropping numerous others.

We also ran this experiment with a higher workload of 603 concurrent clients instead but did not obtain significant results. There was a tremendously high variability in the performance and the success rate of the cross-shard transactions. This is due to the fact that a cross-shard transaction can fail at multiple different stages (preparing, committing, reference creation).

Table 6.1: Phase contribution comparison between fast ($\leq 5s$) and slow ($>5s$) cross-shard transactions. Percentages are computed per transaction.

	$\leq 5s$		$>5s$	
	Average (%)	P95 (%)	Average (%)	P95 (%)
reference_create	5.81	10.59	0.22	0.32
begin_tc	1.65	2.32	0.06	0.08
prepare	84.44	92.21	99.41	99.59
commit	8.10	10.52	0.30	0.31

Table 6.2: Phase latency comparison between fast ($\leq 5s$) and slow ($>5s$) cross-shard transactions.

	$\leq 5s$		$>5s$	
	Average (ms)	P95 (ms)	Average (ms)	P95 (ms)
reference_create	36.12	57.10	38.07	53.35
begin_tc	10.61	12.15	10.54	12.99
prepare	627.48	989.75	16864.91	17254.49
commit	51.11	52.14	51.25	52.74

6.4.3 Breakdown of cross-shard transactions phase latencies

In order to gain insights on the specific cross-shard transaction induced delays, we implemented an Observer structure which measures, at the level of the coordinator, the duration of each phase. More specifically, it measures the time it takes for such a transaction to be recorded as a new transaction in TC-Raft and the delays of the prepare and commit phases.

We also add an Observer structure at the shard level to measure more fine-grained phase delays within a single-shard such as the time to acquire database connections, and execute the various statements required to prepare or commit a cross-shard transaction.

For these measurements, we removed the background single-shard transactions workload and only use a client that sends the same cross-shard (la-area) / nyc-area) transaction to node 4 (dallas-area) 100 times, waiting 15 seconds between each transactions to allow the system to recover completely then averaging out the results of each transaction.

Figure 6.1 shows that the prepare phase takes between 85% and 99% of the query's lifetime. In order to further investigate this, we measure the latency breakdown per measured phase on the shard side as shown in figure 6.3. The most time consuming phase appears to be the action execution during the prepare phase. This is the phase where the node interacts with the database and it is heavy in SQL queries involving scans and updates.

When running the experiments, we noticed that the first 5 queries finished in a maximum of 1 second while the other 95 queries took around 17s. Figure 6.4 shows this contrast. The 5 first queries

Table 6.3: Cross-shard transaction time breakdown (mean and 95th percentile) recalculate stats, some phases have been removed

	Average (%)	P95 (%)
prepare_action	3.56	3.66
acquire_conn	0.04	0.04
begin_isolation	0.00	0.00
execute_action	95.82	96.01
prepare_transaction	0.12	0.12
propose_commit_raft	0.46	0.75

Table 6.4: Cross-shard phase breakdown for the first and remaining transactions.

	Average (%)		P95 (%)	
	≤5s	>5s	≤5s	>5s
prepare_action	45.43	1.36	47.05	1.37
acquire_conn	0.63	0.01	0.71	0.01
begin_isolation	0.05	0.00	0.07	0.00
execute_action	46.15	98.43	48.10	98.53
prepare_transaction	1.88	0.02	1.97	0.02
propose_commit_raft	5.87	0.18	9.43	0.29

only spend an average of 46% of their time on `execute_action` while the remaining ones spend 98% of their duration on it.

Since the transactions are sent sequentially to the system, resource contention is not a viable explanation. Instead, the database interactions seemed to be increasing in complexity after only 5 cross-shard transactions. More specifically, figure 6.19 shows that one specific interaction with the database (`SearchOperationalIntents`) takes a tremendous amount of time compared to the others starting from the sixth transaction. Factors such as suboptimal database indexing could be the cause of this performance drop. Investigating them is left for future work.

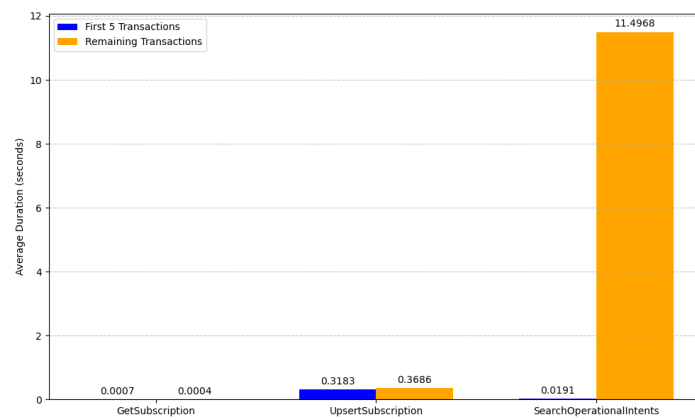


Figure 6.19: Average duration of each of the database interactions for CreateSubscription cross-shard transactions.

Chapter 7

Related and Future Work

7.1 Distributed Databases

Distributed databases like Google Spanner [7] and CockroachDB [8] are of high significance in achieving global consistency with geographical sharding. The former relies on underlying Paxos groups while the latter is based on Raft. Both systems use or have used a Two-Phase Commit (2PC) protocol for transactions involving more than one shard although CockroachDB's protocol has optimized it into protocols called Transaction Pipelining then Parallel Commits which allow for more concurrency of the commits. Spanner's implementation is also more advanced than the traditional 2PC protocol. Cross-group transactions in Spanner are not blocking thanks to the underlying Paxos groups and the use of the TrueTime API to obtain commit timestamps that ensure external consistency. This is possible as Spanner is deployed on Google owned infrastructure, allowing control over GPS receivers and clocks as opposed to other systems designed to run independently of the underlying hardware.

7.2 Blockchains

Given the issue of low transaction throughput in blockchains[24], there have been multiple protocols aiming to improve the performance of cross-shard transaction processing in sharding-based blockchains. The paper "Cross-shard Transaction Processing in Sharding Blockchains" [25] introduces a protocol based on 2PC but designed for batch processing, reducing the number of consensus calls. The authors of "X-Shard: Optimistic Cross-Shard Transaction Processing for Sharding-Based Blockchain" [26] aim to improve the waiting time caused by 2PC by proposing a blockchain system that assigns transactions to shards according to previous patterns to minimize cross-shard interactions. It adopts an optimistic approach that allows parallel sub-transactions processing within their

shards, improving overall throughput.

7.3 Spatial Data storage

Some systems, namely spatial database systems [27], are specifically built to handle data with spatial extents such as geometric and geographical data. They use specialized spatial (or temporal) indexing techniques and are essential in applications such as geographic information systems (GIS).

7.4 Discussion and Requirements

While many systems focus on optimizing the Two-Phase commit (2PC) protocol for performance, there is limited literature on its application over geographically sharded data. Most systems optimize transaction processing within traditional distributed databases or blockchains which shard based on replica placement and data locality without looking into the specific case of geographical data. In this thesis, we aimed to evaluate the effectiveness of sharding based on geographical data while keeping a less optimized version of 2PC.

The current design and implementations meet the majority of the Must Have and Should Have requirements listed in the appendix A and provide a system capable of achieving requests under the 5 seconds timeout up to hundreds of QPS of throughput.

The standards mentioned in the first "Must Have" requirements A.1 require that the DSS provides a strongly consistent view of the airspace to the clients which is an inherent property of Raft-based sharding. Amongst the UTM specific parts of the standards, we only cover the strategic deconfliction part of the API in our implementation for simplicity. Supporting other features such as remote id is a simple matter of extending the API.

Our system is also in accordance with requirements 2, 4, 6 and 7 due to its use of the etcd library and the simplicity of the protocol. It is also self-contained and only needs an external Postgres node running to be deployed. Requirement number 5 was tested in section 6.3.1. Requirements number 3 and 9 should be further tested with more realistic workloads and deployments and is left for future work.

Regarding the "should have" requirements, number 2 is a natural consequence of using Raft: Each Raft group can tolerate the failure of a minority of nodes. However, if a shard is down, its geographical area is not accessible during that time. This shortcoming can be addressed using dynamic sharding which is left as future work.

One important "should have" requirement that is not covered by our design and implementation is number 4: "It is difficult for a member of a particular DSS instance provider to accidentally corrupt the DAR". Our implementation assumes that all commands are deterministically applied.

However, in practice, if a single DSS node's database starts to diverge due to a different version or implementation, fundamental requirements could be broken where a client would get different answers depending on which node it contacts.

Chapter 8

Conclusion

This thesis introduced a Raft-based application-level consensus protocol for the Discovery and Synchronization Service internal synchronization. By partitioning the airspace into geographical shards supported by independent Raft groups, we established a multi-Raft architecture which enhances the performance of a single-Raft based design. Our experiments revealed that while geographical sharding can increase throughput for the single-shard transactions, routing and coordination overhead costs represent areas for further improvement.

As such, future work should focus on integrating Raft request batching to alleviate leader bottleneck issues as well as the limitations mentioned in 5.2.5. Implementing authentication between nodes, system monitoring, an extensive test suite and addressing the lack of support for certain requirements are also important aspects that need to be addressed before the system can move to production.

To fully characterize the proposed system, future measurements should experiment with various Raft parameters such as the number of entries required to trigger snapshots which impact the system's performance. It is also necessary to conduct experiments with more realistic workloads and bigger deployments and further expand measurements on different failure and network delays scenarios where regions can be arbitrarily slow or crashing.

Eventually, more advanced designs should explore dynamic sharding, optimizations for the 2PC protocol inspired by the mentioned related work and alternative routing options.

In conclusion, this thesis provides valuable insights into the characterization of a consensus-based synchronization protocol for the DSS as an addition to the ongoing development effort of this solution.

Bibliography

- [1] F. A. Administration, *Unmanned aircraft system traffic management (utm)*, https://www.faa.gov/uas/advanced_operations/traffic_management, Last updated May 2, 2025, 2025.
- [2] *Standard specification for uas traffic management (utm) uas service supplier (uss) interoperability*, Last updated Mar. 2022, ASTM International. [Online]. Available: <https://www.astm.org/f3548-21.html>.
- [3] *Interuss platform*, <https://interussplatform.org/>, Linux Foundation / InterUSS Project.
- [4] C. Cachin, R. Guerraoui, and L. Rodrigues, *Introduction to Reliable Distributed Programming*, 2nd. Springer, 2011, ISBN: 978-3-642-15259-7. DOI: 10.1007/978-3-642-15260-3. [Online]. Available: <https://doi.org/10.1007/978-3-642-15260-3>.
- [5] L. Lamport, “The part-time parliament”, *ACM Trans. Comput. Syst.*, vol. 16, no. 2, pp. 133–169, May 1998, ISSN: 0734-2071. DOI: 10.1145/279227.279229. [Online]. Available: <https://doi.org/10.1145/279227.279229>.
- [6] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm”, in *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, ser. USENIX ATC’14, Philadelphia, PA: USENIX Association, 2014, pp. 305–320, ISBN: 9781931971102.
- [7] J. C. Corbett et al., “Spanner: Google’s globally distributed database”, *ACM Trans. Comput. Syst.*, vol. 31, no. 3, Aug. 2013, ISSN: 0734-2071. DOI: 10.1145/2491245. [Online]. Available: <https://doi.org/10.1145/2491245>.
- [8] R. Taft et al., “Cockroachdb: The resilient geo-distributed sql database”, in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’20, Portland, OR, USA: Association for Computing Machinery, 2020, pp. 1493–1509, ISBN: 9781450367356. DOI: 10.1145/3318464.3386134. [Online]. Available: <https://doi.org/10.1145/3318464.3386134>.
- [9] Yugabyte, *Raft consensus protocol*, Accessed: 2025-10-16. [Online]. Available: <https://docs.yugabyte.com/preview/architecture/docdb-replication/raft/>.
- [10] G. Zhang et al., “Reaching consensus in the byzantine empire: A comprehensive review of bft consensus algorithms”, *ACM Comput. Surv.*, vol. 56, no. 5, Jan. 2024, ISSN: 0360-0300. DOI: 10.1145/3636553. [Online]. Available: <https://doi.org/10.1145/3636553>.

- [11] M. Castro and B. Liskov, “Practical byzantine fault tolerance and proactive recovery”, *ACM Trans. Comput. Syst.*, vol. 20, no. 4, pp. 398–461, Nov. 2002, ISSN: 0734-2071. DOI: 10.1145/571637.571640. [Online]. Available: <https://doi.org/10.1145/571637.571640>.
- [12] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, “Hotstuff: Bft consensus with linearity and responsiveness”, in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, ser. PODC ’19, Toronto ON, Canada: Association for Computing Machinery, 2019, pp. 347–356, ISBN: 9781450362177. DOI: 10.1145/3293611.3331591. [Online]. Available: <https://doi.org/10.1145/3293611.3331591>.
- [13] L. Lamport, “Fast paxos”, *Distrib. Comput.*, vol. 19, no. 2, pp. 79–103, Oct. 2006, ISSN: 0178-2770. DOI: 10.1007/s00446-006-0005-x. [Online]. Available: <https://doi.org/10.1007/s00446-006-0005-x>.
- [14] I. Moraru, D. G. Andersen, and M. Kaminsky, “There is more consensus in egalitarian parliaments”, in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, ser. SOSP ’13, Farmington, Pennsylvania: Association for Computing Machinery, 2013, pp. 358–372, ISBN: 9781450323888. DOI: 10.1145/2517349.2517350. [Online]. Available: <https://doi.org/10.1145/2517349.2517350>.
- [15] D. Rystsov, *Caspaxos: Replicated state machines without logs*, 2018. arXiv: 1802.07000 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/1802.07000>.
- [16] P. Tennage et al., “Quepaxa: Escaping the tyranny of timeouts in consensus”, in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP ’23, Koblenz, Germany: Association for Computing Machinery, 2023, pp. 281–297, ISBN: 9798400702297. DOI: 10.1145/3600006.3613150. [Online]. Available: <https://doi.org/10.1145/3600006.3613150>.
- [17] Google, *S2 cell hierarchy*, Accessed: 2026-02-19. [Online]. Available: https://s2geometry.io/devguide/s2cell_hierarchy.html.
- [18] InterUSS, *Dss concepts*, Accessed: 2026-02-19. [Online]. Available: <https://github.com/interuss/dss/blob/master/concepts.md>.
- [19] Google, *Golang geo library*, Accessed: 2026-02-19. [Online]. Available: <https://github.com/golang/geo>.
- [20] InterUSS TSC, *Cockroachdb migration investigation*, 2024. [Online]. Available: https://github.com/interuss/tsc/blob/main/investigations/crdb_migration.
- [21] InterUSS TSC, *Cockroachdb mitigation approaches*, 2024. [Online]. Available: https://github.com/interuss/tsc/blob/main/investigations/crdb_migration/%5BInterUSS%5D%20CockroachDB%20mitigation%20approaches.pdf.
- [22] etcd-io, “Raft library”, in *GitHub repository*. [Online]. Available: <https://github.com/etcd-io/raft>.

- [23] InterUSS TSC, *Interuss dss 2024 redesign*, 2024. [Online]. Available: https://github.com/interuss/tsc/blob/main/investigations/crdb_migration/InterUSS%20DSS%202024%20redesign.docx.
- [24] I. S. Rao, M. L. M. Kiah, M. M. Hameed, and Z. A. Memon, “Scalability of blockchain: A comprehensive review and future research direction”, *Cluster Computing*, vol. 27, no. 5, pp. 5547–5570, Feb. 2024, ISSN: 1386-7857. DOI: 10.1007/s10586-023-04257-7. [Online]. Available: <https://doi.org/10.1007/s10586-023-04257-7>.
- [25] Y. Liu, J. Liu, J. Yin, G. Li, H. Yu, and Q. Wu, “Cross-shard transaction processing innbsp;sharding blockchains”, in *Algorithms and Architectures for Parallel Processing: 20th International Conference, ICA3PP 2020, New York City, NY, USA, October 2–4, 2020, Proceedings, Part III*, New York, NY, USA: Springer-Verlag, 2020, pp. 324–339, ISBN: 978-3-030-60247-5. DOI: 10.1007/978-3-030-60248-2_22. [Online]. Available: https://doi.org/10.1007/978-3-030-60248-2_22.
- [26] J. Xu, Y. Ming, Z. Wu, C. Wang, and X. Jia, “X-shard: Optimistic cross-shard transaction processing for sharding-based blockchains”, *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 4, pp. 548–559, Apr. 2024, ISSN: 1045-9219. DOI: 10.1109/TPDS.2024.3361180. [Online]. Available: <https://doi.org/10.1109/TPDS.2024.3361180>.
- [27] R. H. Güting, “An introduction to spatial database systems”, *The VLDB Journal*, vol. 3, no. 4, pp. 357–399, 1994.

Appendix A

Previous Work

Table A.1: TSC's Must Have requirements for a DSS Implementation

Number	Requirement Description
1	- DSS implementation meets ASTM F3548-21 and ASTM F3411-22a requirements.
2	- The implementation can be developed and maintained by the available InterUSS contributors.
3	- Reasonable deployment of the DSS implementation is capable of handling roughly country-scale traffic in the medium term.
4	- The operator of a DSS instance can define which DSS instances are part of the pool in which it is participating.
5	- Failure of one DSS instance in a pool of at least three instances allows the remaining instances to function without a substantial increase in latency.
6	- Deployment is not tied to a single/specific cloud provider.
7	- Each DSS instance must be an equal peer (excluding temporary circumstances such as Raft leaders).
8	- Implementation can be explained to semi-technical regulators and maps to conceptual DSS functionality described in ASTM standards.
9	- USS use of the InterUSS DSS implementation must allow the USS to meet performance requirements.

Table A.2: TSC's Should Have requirements for a DSS Implementation

Number	Requirement Description
1	- Implementation does not depend on non-free/restricted-license products.
2	- Failure of a minority of DSS instances in a pool allows the pool to continue to function.
3	- Initial deployment and ongoing maintenance does not require a large amount of effort or specialized expertise.
4	- It is difficult for a member of a particular DSS instance provider to accidentally corrupt the DAR.
5	- A DSS instance provider can unilaterally improve DSS performance by contributing more resources.
6	- Deployment of a low-traffic DSS instance is relatively inexpensive.

Table A.3: TSC's Could Have requirements for a DSS Implementation

Number	Requirement Description
1	- DSS implementation can interoperate with non-InterUSS DSS implementations with minimal constraints.
2	- Slow failure of any number of DSS instances in a pool of at least three instances allows the pool to continue to function.

Appendix B

Implementation details

B.1 Multi-Raft configuration file

```
shards:
- id: la-area
  participants:
    1: 10.10.10.2:8081
    2: 10.10.10.3:8081
    3: 10.10.10.4:8081
  extents:
    - lat: 33.430379
      lng: -117.603778
    - lat: 34.213404
      lng: -119.450718
    - lat: 34.341050
      lng: -116.543768
    - lat: 33.546606
      lng: -117.227277
- id: dallas-area
  participants:
    4: 10.10.10.2:8082
    5: 10.10.10.3:8082
    6: 10.10.10.4:8082
  extents:
    - lat: 32.525289
      lng: -97.065956
```

```

- lat: 32.960018
  lng: -97.164509
- lat: 32.974717
  lng: -96.603853
- lat: 32.674401
  lng: -96.585748
- id: nyc-area
  participants:
    7: 10.10.10.2:8083
    8: 10.10.10.3:8083
    9: 10.10.10.4:8083
  extents:
    - lat: 40.571547
      lng: -74.317341
    - lat: 40.789207
      lng: -74.290888
    - lat: 40.798092
      lng: -73.811553
    - lat: 40.613772
      lng: -73.695770

```

B.2 Single-Raft parameters

```

var snapshotCatchUpEntriesN uint64 = 10000
conf := raft.Config{
  ID:                c.cluster.id,
  ElectionTick:      20,
  HeartbeatTick:     1,
  Storage:           c.raftStorage,
  MaxSizePerMsg:     1024 * 1024,
  MaxInflightMsgs:   256,
  MaxUncommittedEntriesSize: 1 << 30,
}

```

B.3 Proposals

```

type Proposal struct {
  ID          string          `json:"id"`

```

```

Timestamp    time.Time          'json:"timestamp,omitempty"'
RequestType  string              'json:"request_type"'
Value        []byte              'json:"value"'
ReadOnly     bool                'json:"read_only"'
Parameters   map[string][]byte  'json:"parameters,omitempty"'
}

```

B.4 gRPC Consensus Service

```

service Consensus {
    rpc SyncRaft(stream RaftMessage) returns (Ack);
}

```

```

message RaftMessage {
    string from = 1;
    string to = 2;
    bytes data = 3;
}

```

```

message Ack {
    bool success = 1;
}

```

B.5 gRPC Coordinator Service

```

service Coordinator {
    rpc Prepare(MsgPrepare) returns (MsgPrepareResponse);
    rpc Decision(MsgDecision) returns (MsgDecisionAck);
}

```

```

message MsgPrepare {
    string id = 1;
    bytes data = 2;
}

```

```

message MsgPrepareResponse {
    bytes data = 1;
}

```

```

message MsgDecision {
    string id = 1;
    bool commit = 2;
}

message MsgDecisionAck {
    bytes data = 1;
}

```

B.6 gRPC Owners Service

```

service Owners {
    rpc AssignOwners(MsgAssignOwners) returns (google.protobuf.Empty);
    rpc GetOwners(MsgGetOwners) returns (MsgOwners);
    rpc Execute(MsgProposal) returns (MsgProposalResult);
}

message MsgAssignOwners {
    string id = 1;
    repeated string shards = 2;
}

message MsgGetOwners {
    string id = 1;
}

message MsgOwners {
    repeated string ids = 1;
}

message MsgProposal {
    bytes data = 1;
}

message MsgProposalResult {
    bytes data = 1;
}

```